AI 知识护照盘

项目落地指南（PDF 版）

给第一次做 AI 产品、第一次用 Codex、但想一步一步把项目做出来的人。

这份指南解决什么问题

把“灵感、借鉴、PRD、Codex、MCP、知识整理”串成一条能执行的路线。

帮你分清：哪些东西要学习，哪些东西要提炼，哪些东西要真正落到代码里。

用最少变量把项目先做起来，而不是一上来做成一个过重的 agent OS。

最终目标（请反复记住）

你不是在先做一个“万能 agent 平台”。

你在做的是：一个本地优先的知识编译器 + 一个受控的 AI 接入口。

MVP 先验证：AI 读取 Passport 后，是否更快、更准地理解你。

推荐默认栈：Python + SQLite + 本地文件系统 + 最小 MCP Server + Codex CLI

版本：v1.0 | 日期：2026-04-05

如何使用这份指南

- 如果你完全是小白：请按第 1 章到第 7 章顺序读，不要跳。
- 如果你已经有 PRD：直接重点看第 3 章工具地图、第 4 章技术方案、第 5 章落地步骤。
- 如果你已经开始用 Codex：重点看第 5 章和第 6 章，把大任务切成可验证的小 milestone。
- 如果你已经在想“插件、skill、agent、MCP”的区别：先看第 3 章，再决定接入方式。

你读完后应该能做到

1. 说清楚项目到底是什么，不是什么。
2. 解释每个工具的角色：Codex、AGENTS.md、Skills、MCP、SQLite、Git。
3. 从 0 开始建一个最小仓库，并按周推进到可用的 Passport 原型。
4. 知道什么时候应该继续扩功能，什么时候应该停下来先优化编译质量。

文档地图

章节	你会得到什么
第 1 章	把项目方向想对：你到底在造什么
第 2 章	从借鉴中学什么、改什么、拒绝什么
第 3 章	工具地图：每个工具的角色与常见误解
第 4 章	最适合当前阶段的技术方案
第 5 章	从 0 到 1 的一步一步落地流程
第 6 章	怎样正确使用 Codex 作为 coding agent
第 7 章	项目跑起来后的日常操作与 QA
附录	30 天推进表、仓库骨架、首周清单

第 1 章 先把方向想对：你到底在造什么

一句话定义

正确表述

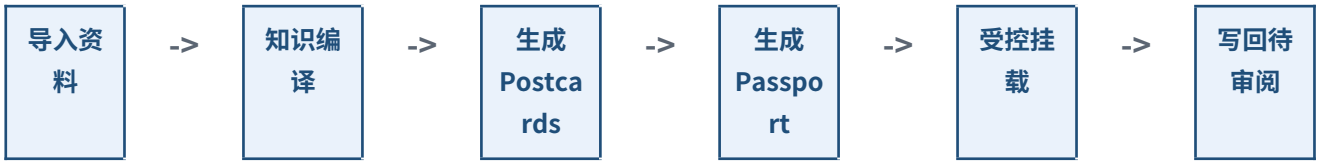
AI 知识护照盘 = 本地优先的知识编译器 + 受控的 AI 接入口。

它把分散文件编译成 Topic / Postcard / Passport，让外部 AI 先读“护照”，再按权限深读。

不要把它理解成下面这些东西：

- 不是通用云笔记。
- 不是一上来就做完整知识图谱。
- 不是重型多 agent runtime。
- 不是让外部 AI 直接写你的主知识库。
- 不是靠 skill 或 plugin 本身当作知识接口。

MVP 的核心链路



真正的成功标准

- AI 是否减少了对你背景的反复追问。
- AI 是否更贴合你的基础水平、当前目标和薄弱点。
- 系统生成的每个高层判断，是否都能追到证据来源。
- 任何外部写回，是否都先进入 review queue。

产品上的收束原则

先做“让 AI 更懂你”，不要做“未来总系统”。

先做 3-5 张代表性 Postcards，不追求第一天就有完整图谱。

先做只读挂载，深读用 Visa 控制，写回必须候选化。

第 2 章 从学习到提炼：哪些该借，哪些不该照搬

你现在最需要学习的，不是“更多框架”，而是三类模式：

- 知识如何持续积累。
- agent 如何在受控条件下持续优化。
- 工程规则如何被 Codex 稳定执行。

2.1 学 llm-wiki：借“编译型知识层”

结论	内容
直接借	raw sources 不可变；维护一个持久的中间层；日常操作分成 ingest / query / lint。
改造后借	把 wiki page 改成 Topic / Postcard / Passport；把直接写正式库改成 candidate + review。

不要照搬	不要把最终产品心智做成“一个 markdown wiki”；也不要让 LLM 拥有正式库写权限。

2.2 学 autoresearch：借“狭窄可写面 + 固定 evaluator”

结论	内容
直接借	人维护 program / 规则文件；agent 只改一个狭窄模块；每次实验都有固定评测与结果账本。
改造后借	把 train.py 换成你的 compiler 模块；把单一 loss 换成主指标 + 护栏指标。
不要照搬	不要做无限自治；不要只用一个指标；不要让 agent 同时改 prompt、评测和实现。

2.3 学 Codex 官方方法学：借“仓库合同”

- 复杂任务先进入 /plan，让 Codex 先读仓库、收集上下文、给出计划。
- 把长期规则写进 AGENTS.md；把当前复杂任务写进 PLANS.md；把重复流程写成 skills。
- 把 done definition 写清楚：build、typecheck、tests、docs 哪些必须通过。
- 让 Codex 一次只做一个 milestone，而不是一次交付整个产品。

你的提炼动作应该怎么做

为每个参考对象都写一份 Take / Adapt / Reject 三栏笔记。

把“灵感”翻译成你自己的对象模型、权限模型和开发约束。

只把真正服务于 MVP 的东西写进仓库规则，其他都放进参考笔记。

第 3 章 工具地图：每个工具到底是什么

工具	它是什么	你现在用它做什么	常见误解
Terminal	发命令的控制台	启动项目、运行 Codex、跑测试	把它当成“高级开发者专属”
Git / GitHub	版本管理与远程仓库	记录改动、回滚、审查 diff	只有团队协作才需要
Python	你当前最省变量的实现语言	写编译器、CLI、本地服务、MCP	语言越多越专业
SQLite	本地数据库文件	存结构化对象与会话记录	必须先上 Postgres 才像产品

Codex CLI	本地 coding agent	读仓库、改代码、跑命令	它就是你的最终产品运行时
AGENTS.md	仓库总规则	定义边界、目录、约束、done	它是知识库或插件
PLANS.md	复杂任务施工图	拆 milestone、记录决策、约束范围	写完一次后就不再更新
Skill	可复用流程包	封装 ingest、review、benchmark 流程	它是对外知识接口
MCP	AI 连接外部系统的协议层	暴露 Passport/Postcards/Tools	它等于“多 agent 系统”
Evaluator	固定评测器	判断编译器是否真的变好	没必要，凭感觉就知道好坏

一个最关键的判断

Skill 不是你的知识库。

Plugin 也不是你的知识协议。

真正适合“让不同 AI 受控读取你的知识”的，是 MCP Server。

3.1 它们之间的关系

AGENTS.md 管规则 -> **PLANS.md 管任务** -> **Skills 管流程** -> **MCP 管接入** -> **SQLite 管结构化数据**

第 4 章 最适合当前阶段的技术方案

推荐默认栈

- 语言：Python
- 原始资料：本地文件系统
- 结构化存储：SQLite
- 应用入口：CLI + 极简本地 HTTP 服务
- 对外接入：最小 MCP Server
- 开发 agent：Codex CLI

为什么先选 Python

- 单语言省脑力，特别适合第一次做 0-1。
- 文件解析、文本处理、本地服务和脚本都很好写。

- 以后做 compiler benchmark 和 evaluator 也很顺。

推荐仓库结构

```
ai-passport/
  AGENTS.md
  PLANS.md
  README.md
  docs/spec/
  docs/reference/
  docs/research/
  data/workspaces/personal/{inbox,raw,exports}/
  app/{domain,storage,ingest,compile,passport,gateway,review,api,mcp}/
  tests/{fixtures,golden,integration}/
  skills/{ingest-source,compile-passport,evidence-trace-review}/
```

核心数据分层

层	放什么	为什么这样放
Raw Files	原始笔记、PDF、网页摘录、项目文档	原文作为 source of truth，不被编译覆盖
Structured DB	Source、Node、Evidence、Postcard、Passport、Visa、Session、Review Candidate	适合权限、审计、候选写回与查询
Exports	passport.json / passport.md / postcard.json	给人看、给 AI 读，且避免外部 AI 直接遍历整库

MCP 最小接口建议

类型	建议对象
Resources	passport://active focus://active postcard://topic/{id}
Tools	request_visa create_writeback_candidate revoke_visa
Prompts	mount_active_passport review_candidate

第一阶段不要做什么

不要先做重型前端。

不要先开放全库搜索。

不要做复杂权限矩阵。

不要做多 agent 协作 runtime。

第 5 章 一步一步落地：从第一个命令到第一个原型

Step 0：先把开发环境准备好

- 安装 Python 3.11+。
- 安装 Git。
- 安装 VS Code（或你熟悉的编辑器）。
- 安装 Codex CLI，并确认你能在终端里运行 codex。
- 新建一个空目录，准备作为项目根目录。

```
mkdir ai-passport
cd ai-passport
git init
python -m venv .venv
source .venv/bin/activate
```

这一步的完成定义

你能进入项目目录。

你能激活虚拟环境。

你能运行 git status。

你能在终端中启动 codex。

Step 1：先建仓库规则，不要急着写业务代码

- 先写 AGENTS.md：说明产品目标、非目标、目录结构、权限边界、done definition。
- 再写 PLANS.md：只放当前阶段的 milestone，不要写成未来三年的 roadmap。
- 把 Borrowed Patterns and Risks 单独记在 docs/reference 或 docs/spec 中。

给 Codex 的第一条任务不是“做完整产品”，而是“做仓库 reconnaissance + M1 schema”。

```
Read AGENTS.md and PLANS.md first.

Milestone: M1 Domain + Storage only.
Do not build UI, MCP, or writeback flows yet.
Create the SQLite schema and domain models for:
```

```
- Workspace, Source, KnowledgeNode, EvidenceFragment, Postcard, Passport, VisaBundle,
MountSession, ReviewCandidate, AuditLog
Return: plan, changed files, tests run, blockers.
```

Step 2：先把数据层站起来

- 优先完成 SQLite schema 和 domain models。
- 把“原始文件”和“结构化对象”分开。
- 至少先保证 Source -> Node -> Evidence -> Postcard -> Passport 这条链可表达。

你现在必须有的对象	为什么
Workspace	区分 personal / work / project 范围
Source	保留原始材料及来源信息
Knowledge Node	承接 topic / project / method / question
Evidence Fragment	让高层结论可回溯
Postcard	让外部 AI 先理解，不必直接读整库
Passport	作为默认入口
Review Candidate	阻止外部 AI 直接污染主库
Audit Log	记录读取、写回、撤回与审阅

Step 3：做导入（ingest），不要先做“智能”

- 先支持 Markdown / TXT / PDF / Web clip 四类来源。
- 导入后统一进 Inbox，再复制或登记到 raw。
- 为每个 Source 记录：标题、类型、来源、导入时间、workspace、隐私级别。

新手最容易犯的错

一上来就想“自动理解一切”。

正确顺序是：先把文件导进来、能看见、能追踪；再谈编译质量。

Step 4：做编译器（compiler）

- 第一版只做最小编译目标：从 Source 生成 Topic / Project / Method / Question 节点。
- 同时抽 Evidence Fragments。
- 再基于节点和证据，生成 3-5 张代表性 Postcards。

- 最后汇总为一个 Passport。

第一版 Passport 至少包含：

- 用户简介（owner summary）
- 当前 Focus Card
- 3-5 张代表性 Postcards
- 主要主题版图（theme map）
- 可见范围说明（默认只读）

Step 5：先做人工验证，不要急着上 live 接入

- 先导出 passport.md + passport.json。
- 拿它分别喂给 GPT / Claude 等 AI，和空白会话做对照。
- 观察：AI 是否减少背景追问，是否更贴合你的基础与目标。

> **Go / No-Go 判断**
>
> 如果 AI 读取 Passport 后仍然“像不认识你”，先继续优化编译器。
>
> 如果已经明显减少背景重讲，再进入 MCP 接入阶段。

Step 6：做 Review Queue 和 Audit Log

- 所有外部 AI 新内容先进入 Review Candidate。
- 用户只能 Accept / Edit then Accept / Reject。
- 记录来源会话、时间、目标对象和 diff。

这一层看似不“炫”，但它是信任边界。没有它，产品很快会失控。

Step 7：做最小 MCP Server

- Resources 先只开放 active passport、active focus、topic postcards。
- Tools 先只开放 request_visa 和 create_writeback_candidate。
- 不要在 P0 就开放整库全文搜索。

```
Resources:
- passport://active
- focus://active
- postcard://topic/{id}

Tools:
- request_visa(topic_id, reason, ttl)
- create_writeback_candidate(target_id, content, kind)
```

Step 8：再做优化，而不是先扩功能

- 只有在端到端链路跑通之后，再开始做 benchmark 与优化循环。
- 学习 autoresearch 的方法：固定 evaluator，限制可写范围，只优化一个模块。
- 让“编译质量变好”成为后续主要任务，而不是“界面更多、概念更多”。

第 6 章 怎样正确使用 Codex

Codex 在这个项目里的正确角色

正确定位

Codex 是帮你造系统、改系统、写测试、跑命令的 coding agent。

Codex 不是你的知识库本体，也不应该替代你的运行时。

推荐工作流

步骤	你要做什么
1	在终端进入项目目录，启动 codex。
2	先输入 /plan，让它先读 AGENTS.md 和 PLANS.md。
3	只给一个 milestone，不给“做完整产品”这种大任务。
4	先看它的计划：改哪些文件、怎么验证、哪些风险。
5	同意后再让它实施。
6	看 diff、跑 tests、让它更新 Documentation.md。

Milestone prompt 模板

```
Read AGENTS.md and PLANS.md first.

Current milestone: <name>.
Goal: <very specific goal>.
Out of scope: <what not to touch>.
Deliverables: <files/modules/tests/docs>.
Validation: run build/typecheck/tests and summarize failures if any.
Return first with a plan before implementing.
```

你每次审 Codex 输出时，只看四件事：

- 是否越权改了不该改的模块。

- 是否真的跑了验证命令。
- 是否把“候选写回”偷换成“直接写主库”。
- 是否更新了共享文档，让后续任务不丢上下文。

第 7 章 系统跑起来后的日常操作

日常动作	系统应该怎么做	你需要人工判断什么
导入文件	进入 inbox；生成 Source 记录；保留原文	这份材料值不值得入库；属于哪个 workspace
运行编译	更新 Node / Evidence / Postcards / Passport	是否过度概括；是否误引证据
导出或挂载 Passport	输出 JSON / Markdown 或通过 MCP 暴露	哪些内容对外可见；哪些需要隐藏
外部 AI 写回	进入 Review Queue，生成 diff 与来源信息	接受、修改后接受，还是拒绝
备份与恢复	保存 DB、exports、logs	多久备份一次；是否需要回滚

一条铁律

原始文件尽量不可变；

外部 AI 默认只读；

任何写回都必须候选化。

第 8 章 常见坑与决策规则

坑	更好的做法
一上来先做厚前端	先证明 Passport 是否让 AI 更懂你，再做 UI
一上来做多 agent	先做单 agent + 明确 milestone
把 skill 当知识库	skill 只封装流程，知识还是存文件系统 + SQLite
让 AI 直接写主库	一律先进入 Review Queue
没有 evaluator 就开始“优化”	先固定 benchmark，再比较改动前后
为了酷炫概念扩协议	先把编译质量和治理做扎实
原始文件和编译结果混在一起	Raw / DB / Exports 三层分开

没有备份	从一开始就建立最小备份习惯

当你拿不准时，问自己两个问题

这个功能是否直接提升“AI 更快、更准地理解我”？

这个功能会不会削弱证据追踪、权限控制或写回治理?

附录 A：30 天推进表

周次	目标	交付物	通过标准
第 1 周	建仓库、规则、数据层	AGENTS.md、PLANS.md、SQLite schema、基础模型	能导入 Source，能持久化结构化对象
第 2 周	做 ingest + compile v0	Node / Evidence / Postcards / Passport	能从真实材料生成 3-5 张 Postcards
第 3 周	做 review + audit	Review Queue、Audit Log、Diff	外部新内容不再直接写主库
第 4 周	做最小 MCP	Resources / Tools / Prompts	AI 能挂载 Passport，并在边界内申请深读

附录 B：首周清单

- 建立项目目录并初始化 Git。
- 安装并确认 Codex CLI 可运行。
- 写 AGENTS.md 和 PLANS.md。
- 完成 SQLite schema。
- 支持最少一种文件导入（推荐 Markdown 或 TXT）。
- 跑通从 Source 到一条最小 Node/Evidence 的链路。

附录 C：你现在最值得记住的 5 句话

- 借思想，不搬外部项目进主仓库。
- Codex 用来开发系统，不是当运行时本体。
- Skill 管流程，MCP 管接入，SQLite 管结构化数据。
- 先让 Passport 有用，再谈更复杂的功能。
- 所有外部写回先候选化，永远不要直接污染主库。

最后的执行建议

如果你今天就开始，请只做两件事：搭好仓库骨架，完成 M1 Domain + Storage。

不要被“还没想清所有未来功能”卡住。这个项目的正确起点，不是更大的愿景，而是更短的闭环。

—— 先证明 AI 更懂你，再继续扩系统。

今天就开始：48 小时行动卡

今天	明天	本周结束前
建项目目录 初始化 Git 确认 codex 可启动	写 AGENTS.md 写 PLANS.md 让 Codex 先做 M1 schema	跑通 Source -> Node -> Evidence 生成第一版 Postcard / Passport

从第一个可验证的小闭环开始，你的项目就已经真正启动。

最后再复习一次：三层结构

Raw	Structured	Access
文件系统 原始笔记、PDF、网页摘录	**SQLite** Source / Node / Evidence / Passport	**MCP** 让外部 AI 先读 Passport，再按签证深读

学会区分原文、结构化对象和对外接口，你就不会在项目早期迷路。